SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Dr. R. Kesavan

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	**5**
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	**5**
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	**5**
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	**5**
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	**5**
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	**5**

7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	**5**
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	**5**
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	**5**
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	**5**

Code of Conduct:

I R. Indhu (192524332) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

R. Indhu

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs

Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. Complete Query Processing Pipeline

Aim

To process an SQL query from input to final result.

Input

SQL query.

Output

Query result.

Pseudocode

```
ALGORITHM QueryProcessing(SQL_Query)
1. START
2. Receive SQL_Query
3. PARSE SQL_Query
     Identify SELECT, FROM, WHERE, JOIN, etc.
     Check syntax
4. IF syntax is invalid THEN
      Display "Syntax Error"
      STOP
   END IF
5. CREATE Query_Tree
     Convert SQL query into relational algebra/tree
6. OPTIMIZE Query_Tree
     Generate possible execution plans
     Estimate cost of each plan
     Select plan with minimum cost
7. EXECUTE selected Query_Plan
     Access required tables/indexes
     Perform selection, join and projection
8. STORE Result
9. DISPLAY Result
10. STOP
```

Flow

SQL Input

↓

Parsing

↓

Query Tree

↓

Optimization

↓

Execution Plan

↓

Execution

↓

Result

Explanation

The DBMS first checks the SQL syntax, converts it into a query tree, selects the least-cost execution plan, executes it, and returns the result.

Q2. Cost-Based Query Optimization

Aim

To select the join order having the lowest estimated I/O cost.

Input

Tables and possible join orders.

Output

Best/lowest-cost join order.

Pseudocode

ALGORITHM CostBasedOptimization(Tables)

1. START

2. Generate all possible join orders

3. Best_Cost ← INFINITY

   Best_Plan ← NULL

4. FOR each Join_Order DO
5. Estimate number of pages for each table
6. Estimate I/O cost of the joins
7. Total_Cost ← Sum of all join costs
8. IF Total_Cost < Best_Cost THEN
9. Best_Cost ← Total_Cost
10. Best_Plan ← Join_Order
11. END IF
12. END FOR
13. Display Best_Plan
14. Display Best_Cost
15. STOP

Simple cost idea

For a basic nested-loop join:

Join Cost ≈ Pages of Outer Table
× Pages of Inner Table

Explanation

The optimizer generates different execution plans, calculates their estimated I/O costs, and selects the plan with the **minimum cost**.

Q3. Nested Loop Join

Aim

To join two relations using the Nested Loop Join algorithm.

Input

Two tables: R and S.

Output

Joined records.

Pseudocode

ALGORITHM NestedLoopJoin(R, S)

1. START
2. FOR each tuple r in R DO

3. FOR each tuple s in S DO
4. IF JoinCondition(r, s) = TRUE THEN
5. OUTPUT (r, s)
6. END IF
7. END FOR
8. END FOR
9. STOP

Time Complexity

If:

R contains M tuples

S contains N tuples

Then:

Time Complexity = O(M × N)

For page-based block nested-loop join, a common I/O cost estimate is:

B(R) + ceil(B(R)/(B-2)) × B(S)

where:

- B(R) = pages of R
- B(S) = pages of S
- B = available buffer pages

Explanation

Each tuple of the first relation is compared with tuples of the second relation.

Q4. Deadlock Detection Using Wait-For Graph

Aim

To detect deadlocks using a **Wait-For Graph (WFG)**.

Concept

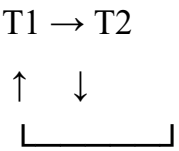
T1 → T2

means:

T1 is waiting for T2.

A cycle indicates a deadlock.

Example:



Pseudocode

```
ALGORITHM DetectDeadlock(WFG)
1. START
2. Mark all transactions as UNVISITED
3. FOR each transaction T DO
4.     IF T is UNVISITED THEN
5.         IF DFS(T) = TRUE THEN
6.             Display "Deadlock Detected"
7.             STOP
8.         END IF
9.     END IF
10. END FOR
11. Display "No Deadlock"
12. STOP

FUNCTION DFS(T)
1. Mark T as VISITING
2. FOR each transaction U connected from T DO
3.     IF U is VISITING THEN
4.         RETURN TRUE
5.     END IF

6.     IF U is UNVISITED AND DFS(U) = TRUE THEN
7.         RETURN TRUE
8.     END IF
9. END FOR
```

10. Mark T as VISITED

11. RETURN FALSE

Explanation

If DFS finds an edge to a transaction that is already **VISITING**, a cycle exists, which means a deadlock exists.

Q5. Basic Two-Phase Locking (2PL)

Aim

To implement locking and releasing according to the Basic 2PL protocol.

Two phases

Growing Phase

↓

Locks can be acquired

↓

Shrinking Phase

↓

Locks can be released

Pseudocode

ALGORITHM Basic2PL(Transaction T)

1. START
2. // Growing Phase
3. FOR each data item X required by T DO
4. Request lock(X)
5. IF lock(X) is available THEN
6. Grant lock(X)
7. ELSE
8. Wait until lock(X) is released
9. END IF
10. END FOR
11. Perform all required operations

12. // Shrinking Phase

13. FOR each locked item X DO

14. Release lock(X)

15. END FOR

16. COMMIT T

17. STOP

Important rule

Once the transaction releases its **first lock**, it cannot obtain any new lock.

Explanation

2PL ensures conflict-serializability by separating locking into growing and shrinking phases.

Q6. Timestamp Ordering with Thomas' Write Rule

Aim

To handle conflicting read/write operations using timestamps and Thomas' Write Rule.

Basic idea

Each transaction receives a timestamp:

TS(T)

Each data item maintains:

Read_TS(X)

Write_TS(X)

Pseudocode

ALGORITHM ThomasWriteRule(T, X, Operation)

1. START

2. IF Operation = READ(X) THEN

3. IF TS(T) < Write_TS(X) THEN

4. ABORT T

5. ELSE

6. Perform READ(X)

7. Read_TS(X) = MAX(Read_TS(X), TS(T))

8. END IF
9. ELSE IF Operation = WRITE(X) THEN
10. IF TS(T) < Read_TS(X) THEN
11. ABORT T
12. ELSE IF TS(T) < Write_TS(X) THEN
13. IGNORE this WRITE
14. // Thomas' Write Rule
15. ELSE
16. Perform WRITE(X)
17. Write_TS(X) = TS(T)
18. END IF
19. END IF
20. STOP

Explanation

Thomas' Write Rule says that if an old transaction tries to perform a write that has already been made obsolete by a newer write, the old write can be **ignored instead of aborting the transaction**.

Q7. ARIES Recovery

Aim

To recover the database after a crash using **Analysis, Redo and Undo** phases.

Pseudocode

ALGORITHM ARIES_Recovery(Log)

1. START
2. // ANALYSIS PHASE
3. Read log from the last checkpoint
4. Identify:

Active transactions

Dirty pages

Committed transactions

5. Build Transaction Table

6. Build Dirty Page Table

7. // REDO PHASE

8. Find the earliest required log record

9. FOR each log record from that point DO

10. IF operation needs REDO THEN

11. Repeat the logged operation

12. END IF

13. END FOR

14. // UNDO PHASE

15. FOR each uncommitted transaction DO

16. Process its log records backward

17. Undo its changes

18. Write compensation log records

19. END FOR

20. Write recovery completion record

21. STOP

ARIES Flow

Crash

↓

Analysis

↓

Redo

↓

Undo

↓

Database Recovered

Explanation

- **Analysis:** Finds active transactions and dirty pages.
- **Redo:** Repeats necessary operations.

- **Undo:** Removes changes made by incomplete transactions.

Q8. Shadow Paging

Aim

To recover a database using shadow and current page tables.

Concept

There are two page tables:

Shadow Page Table → Stable version

Current Page Table → Updated version

Pseudocode

ALGORITHM ShadowPaging

1. START
2. Create Shadow_Page_Table
3. Copy Shadow_Page_Table to Current_Page_Table
4. WHILE transaction is running DO
5. For every page update:
6. Create a new copy of the page
7. Update Current_Page_Table
8. Keep Shadow_Page_Table unchanged
9. END WHILE
10. IF transaction COMMIT succeeds THEN
11. Make Current_Page_Table the new stable table
12. ELSE
13. Discard Current_Page_Table
14. Restore Shadow_Page_Table
15. END IF
16. STOP

Explanation

The original shadow table remains unchanged. If failure occurs, the system discards the current table and uses the shadow table to restore the previous consistent database state.

Q9. Immediate Update Recovery – UNDO/REDO

Aim

To recover the database after a crash using an immediate-update log.

Log format

<T1, X, old_value, new_value>

Pseudocode

ALGORITHM ImmediateUpdateRecovery(Log)

1. START
2. Read the log from the beginning
3. Identify:
   Committed transactions
   Uncommitted transactions
4. // REDO committed transactions
5. FOR each committed transaction T DO
6. FOR each log record of T DO
7. Write new_value to database
8. END FOR
9. END FOR
10. // UNDO uncommitted transactions
11. FOR each uncommitted transaction T DO
12. Read its log records in reverse order
13. Restore old_value
14. END FOR
15. Write recovery completion record
16. STOP

Explanation

After a crash:

Committed transactions → REDO

Uncommitted transactions → UNDO

This restores the database to a consistent state.

Q10. Database Connectivity Manager

Aim

To design a database connectivity manager with connection pooling, query execution, and rollback.

Pseudocode

ALGORITHM DatabaseManager

1. START
2. Create Connection Pool
3. Add database connections to Pool
4. FUNCTION ExecuteQuery(Query)
5. Get a connection from Pool
6. TRY
7. Start Transaction
8. Execute Query
9. IF Query is successful THEN
10. COMMIT transaction
11. Return Result
12. ELSE
13. ROLLBACK transaction
14. Return Error
15. END IF
16. CATCH DatabaseException
17. ROLLBACK transaction
18. Display Error Message
19. FINALLY
20. Return connection to Pool
21. END TRY
22. END FUNCTION
23. STOP

Flow

Request

↓

Get Connection

↓

Start Transaction

↓

Execute Query

↓

Success? ── Yes → COMMIT

|

No

↓

ROLLBACK

↓

Return Connection

Explanation

Connection pooling reuses existing database connections. A successful query is committed, while an error causes a rollback to maintain database consistency.